

Data Structures: Assignment-4

Name: Suraj Shah

Admission Number: I24AI012

Q1: Given a linked list of n nodes and an integer k, write a function to rotate the linked list counter clockwise by k nodes.

Code:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *createNode(int num) {
    struct Node *newNode=(struct Node*)malloc(sizeof(struct Node));
    newNode->data=num;
    newNode->next=NULL;
    return newNode;
}

void rotateLinkedList(struct Node **head, int k) {
    if(*head==NULL || k==0) {
        return;
    }
    struct Node *current=*head;
    while(current->next!=NULL) {
        current=current->next;
    }
    current->next=*head;
    current=*head;
    for(int i=1; i<k; i++) {
        current=current->next;
    }
    *head=current->next;
    current->next=NULL;
}

void displayList(struct Node *head) {
    while(head!=NULL) {
        printf("%d->",head->data);
        head=head->next;
    }
}
```

```

    printf("NULL\n");
}
int main() {
    struct Node *head=NULL;
    struct Node *temp=NULL;
    int n,num;
    printf("Enter the number of nodes: ");
    scanf("%d",&n);
    for(int i=0; i<n; i++) {
        printf("Enter the value of node %d: ",i+1);
        scanf("%d",&num);
        struct Node *newNode=createNode(num);
        if(head==NULL) {
            head=newNode;
        }
        else {
            temp->next=newNode;
        }
        temp=newNode;
    }
    printf("Original List:\n");
    displayList(head);
    int k;
    printf("Enter an integer to rotate the linked list counter clockwise: ");
    scanf("%d",&k);
    rotateLinkedList(&head,k);
    printf("Rotated List:\n");
    displayList(head);
}

```

Output:

```

Enter the number of nodes: 5
Enter the value of node 1: 10
Enter the value of node 2: 20
Enter the value of node 3: 30
Enter the value of node 4: 40
Enter the value of node 5: 50
Original List:
10->20->30->40->50->NULL
Enter an integer to rotate the linked list counter clockwise: 3
Rotated List:
40->50->10->20->30->NULL

```

Q2: Given an unsorted linked list of n nodes, remove duplicates from the list.

Code:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *createNode(int num) {
    struct Node *newNode=(struct Node*)malloc(sizeof(struct Node));
    newNode->data=num;
    newNode->next=NULL;
    return newNode;
}

void removeDuplicates(struct Node *head) {
    struct Node *current=head;
    struct Node *temp;
    struct Node *prev;
    while(current!=NULL) {
        prev=current;
        temp=current->next;

        while(temp!=NULL) {
            if(temp->data==current->data) {
                prev->next=temp->next;
                free(temp);
                temp=prev->next;
            }
            else {
                prev=temp;
                temp=temp->next;
            }
        }
        current=current->next;
    }
}

void displayList(struct Node *head) {
    while(head!=NULL) {
        printf("%d->",head->data);
        head=head->next;
    }
    printf("NULL\n");
}

int main() {
```

```

    struct Node *head=NULL;
    struct Node *temp=NULL;
    int n,num;
    printf("Enter the number of nodes: ");
    scanf("%d",&n);
    for(int i=0; i<n; i++) {
        printf("Enter the value of node %d: ",i+1);
        scanf("%d",&num);
        struct Node *newNode=createNode(num);
        if(head==NULL) {
            head=newNode;
        }
        else {
            temp->next=newNode;
        }
        temp=newNode;
    }
    printf("Original List:\n");
    displayList(head);
    removeDuplicates(head);
    printf("List after removing duplicate elements:\n");
    displayList(head);
}

```

Output:

```

Enter the number of nodes: 10
Enter the value of node 1: 1
Enter the value of node 2: 2
Enter the value of node 3: 3
Enter the value of node 4: 4
Enter the value of node 5: 5
Enter the value of node 6: 5
Enter the value of node 7: 4
Enter the value of node 8: 3
Enter the value of node 9: 2
Enter the value of node 10: 1
Original List:
1->2->3->4->5->5->4->3->2->1->NULL
List after removing duplicate elements:
1->2->3->4->5->NULL

```

Q3: Given a singly linked list of n nodes, detect if it contains a loop or not.

Code:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *createNode(int num) {
    struct Node *newNode=(struct Node*)malloc(sizeof(struct Node));
    newNode->data=num;
    newNode->next=NULL;
    return newNode;
}

int checkLoop(struct Node *head) {
    struct Node *slow=head, *fast=head;

    while(slow!=NULL && fast!=NULL && fast->next!=NULL) {
        slow=slow->next;
        fast=fast->next->next;

        if(slow==fast) {
            return 1;
        }
    }
    return 0;
}

void displayList(struct Node *head) {
    while(head!=NULL) {
        printf("%d->",head->data);
        head=head->next;
    }
    printf("NULL\n");
}

int main() {
    struct Node *head=NULL;
    struct Node *temp=NULL;
    int n,num;
    printf("Enter the number of nodes: ");
    scanf("%d",&n);
    for(int i=0; i<n; i++) {
        printf("Enter the value of node %d: ",i+1);
        scanf("%d",&num);
        struct Node *newNode=createNode(num);
```

```

        if(head==NULL) {
            head=newNode;
        }
        else {
            temp->next=newNode;
        }
        temp=newNode;
    }
    if(checkLoop(head)) {
        printf("The linked list contains a loop.");
    }
    else {
        printf("The linked list does not contain any loop.");
    }
}

```

Output:

```

Enter the number of nodes: 5
Enter the value of node 1: 1
Enter the value of node 2: 1
Enter the value of node 3: 3
Enter the value of node 4: 5
Enter the value of node 5: 5
The linked list does not contain any loop.

```

Q4: Write a C/C++ program to implement doubly linked list with the following function

(i) insertAtFirst(&head, new_data): This function should insert the new data/element at the beginning of the linked list.

(ii) insertAtEnd(&head, new_data): This function should insert the new data/element at the end of the linked list

(iii) insertAtMiddle(&head, new_data): This function should insert the new data/element at the middle of the linked list

(iv) insertAfterNode(&head, given_node, new_data): This function should insert the new data/element after the given node in the linked list.

Example: Suppose, you want to insert 60 after node 40 in the given linked list 10 → 20 → 30 → 40 → 50, the updated linked list will be

10 → 20 → 30 → 40 → 60 → 50\

(v) display(&head): This function should display the content of the linked list

Code:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
    struct Node *prev;
};

struct Node *createNode(int new_data) {
    struct Node *newNode=(struct Node*)malloc(sizeof(struct Node));
    newNode->data=new_data;
    newNode->next=NULL;
    newNode->prev=NULL;
    return newNode;
}

void insertAtFirst(struct Node **head, int new_data) {
    struct Node *newNode=createNode(new_data);
    newNode->next=*head;
```

```

    if(*head!=NULL) {
        (*head)->prev=newNode;
    }

    *head=newNode;
    printf("Inserted %d at the beginning\n",new_data);
}

void insertAtEnd(struct Node **head, int new_data) {
    struct Node *newNode=createNode(new_data);
    if(*head==NULL) {
        *head=newNode;
        printf("Inserted %d at the end\n",new_data);
        return;
    }

    struct Node *temp=*head;
    while(temp->next!=NULL) {
        temp=temp->next;
    }

    temp->next=newNode;
    newNode->prev=temp;
    printf("Inserted %d at the end\n",new_data);
}

struct Node* insertAtMiddle(struct Node *head, int new_data)
{
    struct Node *newNode = createNode(new_data);
    int count =0;
    struct Node *ptr = head;
    while(ptr != NULL)
    {
        ptr = ptr->next;
        count++;
    }
    count /= 2;
    int n = 1;
    ptr = head;
    while(n < count)
    {
        ptr = ptr->next;
        n++;
    }
    newNode->prev = ptr;
    newNode->next = ptr->next;
    if(newNode->next != NULL)
    {
        ptr->next->prev = newNode;
    }
}

```



```

    }
    ptr->next = newNode;
    printf("Inserted %d at the middle.\n",new_data);
    return head;
}

void insertAfterNode(struct Node **head, struct Node * given_node, int new_data) {
    if(given_node==NULL) {
        printf("The given node cannot be null.\n");
        return;
    }

    struct Node *newNode=createNode(new_data);
    newNode->next=given_node->next;
    newNode->prev=given_node;

    if(given_node->next!=NULL) {
        given_node->next->prev=newNode;
    }
    given_node->next=newNode;

    printf("Inserted %d after node %d\n",new_data,given_node->data);
}

void display(struct Node *head) {
    if(head==NULL) {
        printf("The list is empty\n");
        return;
    }

    struct Node *temp=head;
    while(temp!=NULL) {
        printf("%d",temp->data);
        if(temp->next!=NULL) {
            printf("<->");
        }
        temp=temp->next;
    }
    printf("\n");
}

int main() {
    struct Node *head=NULL;

    insertAtFirst(&head, 10);
    display(head);

    insertAtEnd(&head, 20);
    display(head);
}

```

```

insertAtEnd(&head, 40);
display(head);

insertAtEnd(&head, 50);
display(head);

head=insertAtMiddle(head, 30);
display(head);

insertAfterNode(&head, head->next->next->next, 60);
display(head);
}

```

Output:

```

Inserted 10 at the beginning
10
Inserted 20 at the end
10<->20
Inserted 40 at the end
10<->20<->40
Inserted 50 at the end
10<->20<->40<->50
Inserted 30 at the middle.
10<->20<->30<->40<->50
Inserted 60 after node 40
10<->20<->30<->40<->60<->50

```